Two-Stage RAG: Baseline Dense Retrieval vs. Cross-Encoder Re-Ranking

Course: Large Language Models & Multi-Agent Systems (LMA): Assignment 1

Author: Naveen Mishra | **Date:** October 2026

GitHub Repository: https://github.com/Mveen3/lma-a1

Primary Deliverable: notebooks/RAG_Baseline_vs_Reranking.ipynb

---

1. Introduction & Dataset Rationale

In standard Retrieval-Augmented Generation (RAG) pipelines, dense bi-encoders are typically used to fetch candidate passages because vector search is fast and easily indexed offline. However, bi-encoders frequently stumble on queries containing subtle constraints, negative conditions, or heavy lexical overlap with irrelevant passages. This report evaluates a two-stage retrieval pipeline that pairs an initial dense bi-encoder with a cross-encoder re-ranker, demonstrating how joint self-attention fixes common retrieval failures and improves the quality of downstream LLM generation.

1.1 Dataset Selection: Catan Rulebook & Almanac

Per the assignment instructions (which suggest technical policies, legal documents, or an in-depth board game rulebook), I chose the official **Catan (5th Edition) Game Rules and Almanac** (`data/catan_base_rules_2020.pdf`, 16 pages). Using a sentence-boundary-aware chunker with a 500-character window and 100-character overlap, the PDF was split into 160 distinct chunks (average length: 422.8 characters).

I specifically selected this rulebook for four practical reasons:

1. **Split Rule Hierarchy:** The document separates general gameplay rules (Pages 3–7) from the alphabetical reference Almanac (Pages 8–15). Basic trading mechanics are introduced early, but strict prohibitions (e.g., zero-card trades or trading like-for-like resources) are tucked away in the Almanac.
2. **High Lexical Overlap (Keyword Traps):** The word *"trade"* appears in dozens of chunks covering domestic trading, maritime 4:1 trading, harbor 2:1 and 3:1 trades, and turn sequences. A bi-encoder matches heavily on the keyword itself rather than the intent of the query.
3. **Negative Constraints & Corner Cases:** Game rules rely heavily on negative qualifiers (e.g., *"you may not give away cards"*, *"may not trade like resources"*) and corner-case exceptions (e.g., breaking Longest Road via an intersecting settlement). Bi-encoders struggle to distinguish negative constraints from positive assertions in vector space.
4. **Clear, Binary Ground Truth:** Unlike subjective policies, game rules provide clear, unambiguous answers. A move is either legal or illegal. This makes it straightforward to spot when an LLM is hallucinating or missing a key restriction due to bad retrieval.

1.2 Implementation Setup

Both pipelines run on the exact same 160-chunk corpus:

- **Bi-Encoder:** `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional dense vectors, normalized for cosine similarity).
- **Cross-Encoder:** `cross-encoder/ms-marco-MiniLM-L-6-v2` (re-ranks Top-10 candidates).
- **Generative LLM:** Google Gemini (`gemini-flash-lite-latest`) accessed through the Google GenAI SDK.
- **Prompting:** Both pipelines supply their Top-3 chunks into an **identical prompt template** that instructs Gemini to act as an authoritative Catan judge and rely strictly on the provided context without outside speculation.

2. Architecture Comparison: Bi-Encoder vs. Cross-Encoder

The trade-off between Bi-Encoders and Cross-Encoders comes down to where token interactions happen: in an independent vector space or inside a joint self-attention matrix.

Architectural Property	**Bi-Encoder** (`all-MiniLM-L6-v2`)	**Cross-Encoder** (`ms-marco-MiniLM-L-6-v2`)
Input Format	Query q and document d encoded separately: $\mathbf{u} = f_\theta(q)$, $\mathbf{v} = f_\theta(d)$.	Query and document concatenated together: [CLS] $\circ$ q $\circ$ [SEP] $\circ$ d $\circ$ [SEP].
Attention Mechanism	**Isolated self-attention**: Query tokens never attend to document tokens during encoding.	**Joint all-to-all cross-attention**: Every query token attends to every document token across all layers.
Scoring Function	Vector similarity: $S(q,d) = \cos(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v}$.	Classification head on pooled [CLS] token: $S(q,d) = \mathbf{W}\mathbf{h}_{\text{CLS}} + b$.
Offline Indexing	**Yes**: Document embeddings are precomputed offline and stored in vector indexes.	**No**: Every query-document pair must pass through the transformer together at runtime.
Computational Cost	$\mathcal{O}(L_q^2) + \mathcal{O}(N \cdot D)$ (sub-millisecond dot products).	$\mathcal{O}(K \cdot (L_q + L_d)^2)$ where K is the number of candidates.
Main Bottleneck	**Information Bottleneck**: Compressing hundreds of words into 384 numbers loses syntactic nuance and negations.	Expensive forward passes; cannot scale to large collections at query time.

Table 1: Architectural comparison between dense Bi-Encoder retrieval and Cross-Encoder re-ranking.

2.1 Why Bi-Encoders Fail on Nuanced Queries

In a Bi-Encoder, the interaction between query and document is reduced to a single dot product between two fixed vectors. Because passage representations are generated without any knowledge of the future query, the encoder has to squeeze every semantic nuance of the passage into 384 dimensions. When a query contains a negation like *"forbidden"* or a modifier like *"without receiving anything"*, the vector search matches general topic keywords (e.g., *"trade"*, *"cards"*) rather than the constraint itself.

In contrast, a Cross-Encoder passes both sequences through 6 transformer layers together. When the model processes the query word *"forbidden"*, self-attention heads directly route attention weights between *"forbidden"* and passage tokens like *"may not trade like resources"*. This token-to-token interaction allows the Cross-Encoder to catch exact qualifiers and negative constraints that cosine similarity misses entirely.

3. Qualitative & Rank Shift Analysis

To stress-test both retrieval pipelines, I designed four test cases targeting known retrieval failure modes: Keyword Traps, Negation/Constraints, Corner Cases, and Document-Specific Buried Rules.

3.1 Summary of Rank Shifts Across Evaluated Queries

Table 2 records the exact rank movements of critical target chunks from my live notebook execution.

Case ID	Query	Chunk	Base	Re-Rank	Shift	Impact on LLM Answer
CASE-1	*"Can a player give away cards without receiving anything in trade?"*	Chunk 127 (P14)	#6	**#2**	**+4**	Promoted explicit prohibition clause into Top-3; eliminated vague inferences.
CASE-2	*"Is a player forbidden from trading 2 sheep cards for 1 sheep card?"*	Chunk 127 (P14)	#10	**#1**	**+9**	Baseline lacked rule; guessed vaguely. Re-ranking quoted rule verbatim.
CASE-3	*"Can an opponent break your Longest Road by building a settlement on it?"*	Chunk 77 (P9) Chunk 57 (P7)	#1 #5	**#1** **#2**	0 **+3**	Preserved primary rule at #1; promoted victory point context into Top-3.
CASE-4	*"What happens if bank runs out of resource cards on a roll?"*	Chunk 92 (P10) Chunk 22 (P4)	#10 #9	**#2** **#3**	**+8** **+6**	Promoted joint supply shortage and yield context into Top-3.

Table 2: Empirical rank shifts and their direct impact on Gemini LLM answer quality.

3.2 Detailed Walkthrough: Case 2 (Negation & Constraints)

- **Question:** *"Is a player forbidden from trading 2 sheep cards for 1 sheep card with another player?"*
- **Ground Truth:** Catan Rulebook Page 14 (Almanac: Trade): *"You may not trade like resources (e.g., 2 wool for 1 wool)."*
- **Baseline Behavior:** The Bi-Encoder scored generic trade chunks highly (Chunks 24, 59, 126 at ranks #1–#3), while burying the governing prohibition chunk (Chunk 127) at **Rank #10**. Consequently, Gemini never saw the official rule.
- **Baseline LLM Output (Dense Bi-Encoder):**

  > *"Official Ruling: Yes. According to the rules, you cannot trade matching resources (such as trading matching resources with another player)."*

  Observation: While inferring the direction from context, the model repeated the vague phrase *"matching resources"* without the rulebook example. In my earlier tests without strict prompts, it even hallucinated analogies like *"trading 3 ore for 1 ore"*.
- **Re-Ranked Behavior:** The Cross-Encoder linked *"forbidden"* with *"may not trade like resources"*, boosting Chunk 127 from **Rank #10 directly to Rank #1** (+9 positions).
- **Re-Ranked LLM Output (Cross-Encoder Pipeline):**

  > *"Official Ruling: Yes. According to the rules, "You may not trade like resources (e.g., 2 wool for 1 wool)."*"

  Observation: With Chunk 127 at Rank #1, the LLM quoted the official rule verbatim, providing an authoritative, grounded ruling without hesitation.

3.3 Detailed Walkthrough: Case 1 (Keyword Trap / Distractor)

- **Question:** *"Can a player give away cards without receiving anything in trade?"*
- **Ground Truth:** Catan Rulebook Page 14 (Almanac: Trade): *"You may not give away cards."*
- **Analysis:** The dense bi-encoder was distracted by heavy lexical overlap with chunks discussing open domestic card trades (Chunks 59, 24, 126). Chunk 127 sat outside the top-3 at **Rank #6**. The Cross-Encoder successfully promoted Chunk 127 to **Rank #2** (+4 positions), ensuring that the explicit prohibition clause was present in the context fed to the LLM.

3.4 Walkthrough: Case 3 & Case 4 (Corner Cases & Multi-Chunk Context)

- **Case 3 (Breaking Longest Road):** The bi-encoder already placed the primary rule (Chunk 77, Page 9) at Rank #1. However, the cross-encoder also promoted Chunk 57 (Page 7 victory point implications) from Rank #5 to Rank #2, providing the LLM with complete context to explain both road continuity and victory point redistribution.
- **Case 4 (Bank Resource Exhaustion):** Answering what happens when resource cards run out requires two distinct rules: the production yield rule (Chunk 22, Page 4) and the bank exhaustion clause (Chunk 92, Page 10). The bi-encoder buried these at ranks #9 and #10. The cross-encoder promoted both into Top-3 (Chunk 92 $\rightarrow$ #2, Chunk 22 $\rightarrow$ #3), enabling an accurate, comprehensive answer.

4. Latency Profiling & Computational Overhead

To evaluate the operational cost of re-ranking, I ran a benchmark of 30 repeated retrieval passes using `time.perf_counter()` on an AMD Ryzen 7 7445HS CPU and NVIDIA RTX 3050 GPU.

Pipeline Stage	Mean Latency	Std Dev	Complexity	Primary Operation
Pipeline 1: Baseline	**5.47 ms**	$\pm$ 1.09 ms	$\mathcal{O}(L_q^2) + \mathcal{O}(N \cdot D)$	Query embed ($\approx$ 5.0 ms) + dot product ($<$ 0.5 ms)
Pipeline 2: Re-Ranking	**22.23 ms**	$\pm$ 4.05 ms	$\mathcal{O}(K \cdot (L_q + L_d)^2)$	Top-10 dense retrieval + Cross-Encoder forward pass
Cross-Encoder Overhead (Δ)	**16.76 ms**	$\pm$ 4.19 ms	$\mathcal{O}(K \cdot L_{\text{concat}}^2)$	10 sequence pairs through 6 transformer layers

Table 3: Empirical latency benchmark measured over 30 evaluation passes.

4.1 Is the Latency Trade-Off Worth It?

From Table 3, adding a Cross-Encoder increases retrieval time by roughly 4.1$\times$ (from 5.47 ms to 22.23 ms). However, looking at the entire RAG request lifecycle:

- Generative LLM generation (time-to-first-token plus token streaming) typically takes between **400 ms and 1,200 ms**.
- The 16.76 ms overhead from the Cross-Encoder represents **less than 2–3% of the total user-facing latency**.
- In return for that negligible 17 ms delay, the pipeline eliminates severe failure modes, boosts critical chunks from rank #10 to rank #1, and prevents hallucinations in official rulings. In production RAG systems, this is an outstanding trade-off.

5. Scaling Analysis: Why Not Cross-Encode the Entire Corpus?

A natural question arises: *If Cross-Encoders are so much more accurate than Bi-Encoders, why not eliminate the Bi-Encoder entirely and score every document in the collection using the Cross-Encoder from the very start?*

The answer lies in two fundamental engineering barriers: **pre-computation limits** and **quadratic attention complexity**.

5.1 1. The Pre-Computation Barrier

In a Bi-Encoder pipeline, document embeddings $\mathbf{v}_i = f_\theta(d_i)$ are completely decoupled from any query q. One can pre-compute embeddings for millions of passages offline, write them to disk, and build sub-linear approximate nearest neighbor (ANN) indexes (such as HNSW or IVF-PQ). When a user query arrives at runtime, I only need to encode the query once and perform cheap vector dot products.

With a Cross-Encoder, the input is the concatenated pair $[\text{CLS}] \circ q \circ [\text{SEP}] \circ d_i \circ [\text{SEP}]$. Because the sequence contains both q and d_i, **no document representation can be pre-computed offline**. Every single document in the corpus must be passed through the entire neural network alongside the query at runtime.

5.2 2. Asymptotic Complexity Comparison

Let N be the total number of chunks in the corpus, K be the number of initial candidates ($K = 10 \ll N$), and L be the sequence length:

$$\text{Full Cross-Encoder Complexity} = \mathcal{O}\left(N \cdot (L_q + L_d)^2 \cdot L_{\text{layers}}\right) \tag{1}$$

$$\text{Two-Stage Funnel Complexity} = \underbrace{\mathcal{O}\left(L_q^2\right)}_{\text{Query Embed}} + \underbrace{\mathcal{O}\left(N \cdot D\right)}_{\text{Vector Dot Product}} + \underbrace{\mathcal{O}\left(K \cdot (L_q + L_d)^2 \cdot L_{\text{layers}}\right)}_{\text{Cross-Encoder on Top-}K\text{ only}} \tag{2}$$

5.3 3. Quantitative Scaling Projections

Based on my measured benchmark of 1.68 ms per candidate pair on the Cross-Encoder:

Corpus Chunks (N)	Bi-Encoder	Two-Stage ($K = 10$)	Full Cross-Encoder	Feasibility / User Impact
160 (Catan Corpus)	5.5 ms	**22.2 ms**	$\approx$ 268 ms	Noticeable 0.27s hesitation
10,000 (Small Manual)	6.0 ms	**22.7 ms**	$\approx$ 16.8 seconds	**Unusable for live search**
100,000 (Enterprise KB)	8.8 ms	**25.5 ms**	$\approx$ 168 seconds (2.8 min)	**Request timeout / failure**
1,000,000 (Web-scale)	18.0 ms	**35.2 ms**	$\approx$ 28 minutes	**Completely impossible**

Table 4: Estimated query latencies showing why full-corpus Cross-Encoding fails at scale.

As shown in Table 4, running a Cross-Encoder over even 10,000 chunks results in an unviable 16.8-second delay per search. The two-stage architecture solves this by acting as a funnel: the Bi-Encoder provides high-recall filtering in sub-6ms ($N \to 10$), and the Cross-Encoder provides high-precision re-ranking on just those 10 candidates in ≈ 17 ms.

6. Conclusion

This investigation demonstrates the concrete strengths and limitations of both retrieval architectures:

1. **Bi-Encoders excel at throughput but miss nuance:** While fast (5.47 ms), vector dot products suffer from an information bottleneck that blurs negative constraints and keyword traps, burying vital rule chunks as low as Rank #10.
2. **Cross-Encoders restore precision:** Joint cross-attention successfully promoted buried chunks (e.g., Chunk 127 jumping from Rank #10 to #1), enabling the LLM to provide verbatim, grounded rulings.
3. **The Two-Stage pattern is essential for scale:** Full-corpus cross-encoding is computationally intractable for any corpus larger than a few hundred passages. A cascaded funnel (Bi-Encoder Top-10 $\to$ Cross-Encoder Top-3) achieves the precision of full cross-attention while maintaining a total retrieval overhead of just 22 ms.

All code, chunks, prompts, and evaluation outputs are preserved and reproducible in my accompanying repository: https://github.com/Mveen3/lma-a1.